Московский государственный университет имени М.В. Ломоносова

Факультет вычислительной математики и кибернетики

Кафедра автоматизации систем вычислительных комплексов

Ефанов Михаил Михайлович

Алгоритм управления перегрузкой на основе пропускной способности и задержки в условиях применения пакетной балансировки

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Научный руководитель:
д.ф.-м.н., профессор, чл.-корр. РАН Смелянский Руслан Леонидович

Научный консультант:
к.ф.-м.н., ассистент Степанов Евгений Павлович

Москва, 2026

Аннотация

Работа посвящена повышению эффективности алгоритма управления перегрузкой BBR в условиях пакетной балансировки. Актуальность обусловлена широким применением балансировки нагрузки, приводящей к переупорядочиванию пакетов, что классические алгоритмы ошибочно интерпретируют как перегрузку. Цель — разработка модификации BBR, устойчивой к указанным эффектам. Проведен обзор версий BBR и существующих решений (TCP-NCR, Lazy TCP, MSwift), показавший неприменимость последних для модификации BBR. Предложено решение: замена фиксированного порога быстрой переотправки пакета динамическим значением, вычисляемым на основе оценки пропускной способности и задержки соединения. Эксперименты в симуляторе NS-3 подтвердили, что предложенная модификация повышает полезную пропускную способность в асимметричных условиях. Однако показано, что ни базовая, ни модифицированная версии не решают проблему достижения суммарной пропускной способности альтернативных путей.

Оглавление

1. Введение . . . 3
   1.1. Актуальность работы . . . 3
   1.2. Цель и задачи работы . . . 6
   1.3. Структура работы . . . 7
2. Обзор литературы . . . 8
   2.1. Обзор версий BBR . . . 8
      2.1.1. BBRv1 . . . 8
      2.1.2. BBRv2 . . . 12
      2.1.3. BBRv3 . . . 13
      2.1.4. Выбор версии BBR для дальнейшего исследования . . . 14
   2.2. Обзор существующих решений для работы алгоритмов управления перегрузкой в условиях пакетной балансировки . . . 15
      2.2.1. TCP-NCR . . . 16
      2.2.2. Lazy TCP (LTCP) . . . 17
      2.2.3. MSwift . . . 17
      2.2.4. Вывод . . . 18
3. Адаптация BBR к условиям пакетной балансировки . . . 20
4. Экспериментальное исследование работы BBR в условиях пакетной балансировки . . . 22
   4.1. Топология сети . . . 22
   4.2. Экспериментальное исследование скорости передачи данных в условиях симметричных маршрутов при управлении базовым алгоритмом . . . 23
   4.3. Экспериментальное исследование скорости передачи данных в условиях асимметричных маршрутов . . . 26
5. Заключение . . . 32
Литература . . . 33

1. Введение

1.1. Актуальность работы

Современные компьютерные сети характеризуются высокой динамикой нагрузки, неоднородностью каналов и все более широким применением многопутевой передачи. Трафик одного транспортного соединения может проходить по нескольким альтернативным маршрутам — как из-за особенностей маршрутизации, так и из-за применения механизмов балансировки нагрузки. Балансировка может выполняться по потокам, то есть пакеты одного транспортного соединения всегда отправляются на один маршрут. Также балансировка может выполняться на уровне отдельных пакетов (per-packet load balancing, packet spraying), когда последовательные пакеты одного и того же соединения целенаправленно распределяются по разным путям. Такая стратегия потенциально повышает использование доступной пропускной способности совокупности путей и улучшает устойчивость передачи к локальным перегрузкам и отказам, но одновременно меняет свойства наблюдаемых транспортным агентом сигналов состояния сети, таких как RTT и дублирующие ACK [1].

Основные понятия и термины. Прежде чем переходить к детальному рассмотрению проблемы, необходимо определить ключевые понятия, которые будут использоваться в работе. Под **транспортным соединением** понимается логический канал связи между двумя сетевыми приложениями, устанавливаемый протоколом транспортного уровня. **Пропускная способность транспортного соединения** - максимальная скорость, с которой возможно передавать данные через соединение. **Пакетная балансировка** (packet spraying) — это частный случай балансировки, при котором решение о выборе маршрута принимается для каждого отдельного пакета, а не для всего потока в целом (то есть для всех пакетов одного транспортного соединения). Такой подход позволяет максимально эффективно использовать доступные сетевые ресурсы, однако создает серьезные проблемы для алгоритмов управления перегрузкой, которые изначально проектировались для работы в условиях однопутевой передачи [2]. В работе рассматривается транспортный уровень сетевого стека протоколов. На этом уровне данные для отправки поделены на сегменты, однако балансировка в сети реализована на сетевом уровне, где данные находятся в пакетах, поэтому в работе будет использоваться термин “пакет”, термин “сегмент” будет использоваться в случае если это оправдано.

Задержка и ее составляющие. Важнейшим понятием для алгоритмов управления перегрузкой является время кругового обхода пакета (Round-Trip Time, RTT). Под RTT понимается временной интервал между моментом отправки пакета отправителем и моментом получения подтверждения от получателя. Этот интервал складывается из нескольких компонентов: времени распространения сигнала по физическим каналам связи, времени обработки пакетов в промежуточных сетевых устройствах (пакетизация) и времени ожидания в очередях на этих устройствах (буферизация). Задержка распространения и пакетизации неизменны, все изменения задержки связаны с буферизацией пакетов. В работе увеличение очереди и увеличение задержки отождествляются.

Пропускная способность и узкое место. Пропускная способность (bandwidth) канала связи — это максимальное количество данных, которое может быть передано по каналу в единицу времени. Важным понятием является “узкое место” (bottleneck) — участок сети с наименьшей пропускной способностью на пути передачи данных. Именно пропускная способность узкого места (`BtlBw` — Bottleneck Bandwidth) определяет максимальную скорость, с которой данные могут передаваться от отправителя к получателю в отсутствие других ограничений. Алгоритм BBR как раз и ориентирован на то, чтобы оценить `BtlBw` и `RTprop` и на основе этих оценок управлять скоростью передачи.

Сигналы перегрузки и управление потоком. Классические алгоритмы управления перегрузкой TCP [3] исторически опираются на интерпретацию потери пакета (loss-based) как основного признака перегрузки. Эта эвристика была разумной в условиях малых буферов: если буфер маршрутизатора переполняется, пакет отбрасывается, и отправитель, обнаружив потерю (например, по тайм-ауту или по получению трех дублирующих ACK), уменьшает скорость отправки, тем самым снижая нагрузку на сеть. При больших буферах loss-based управление перегрузкой склонно формировать устойчивые очереди и увеличивать задержки, а при потерях, не связанных с перегрузкой — неоправданно снижать скорость, недоиспользуя доступную емкость канала.

Дублирующие ACK и их значение. Для понимания проблем, возникающих при пакетной балансировке, важно разобраться с механизмом дублирующих ACK. В протоколе TCP получатель отправляет подтверждение (ACK) с номером следующего ожидаемого байта. Если получатель принимает сегмент, номер которого больше ожидаемого (то есть между ожидаемым и полученным сегментами есть пропуск), он немедленно отправляет ACK с тем же номером ожидаемого байта, что и предыдущий — такой ACK называется **дублирующим**. Получение трех дублирующих ACK подряд интерпретируется отправителем как сигнал о том, что сегмент с соответствующим номером был потерян, и запускается механизм `Fast Retransmit`: отправитель немедленно повторно отправляет потерянный сегмент, не дожидаясь тайм-аута, и уменьшает окно перегрузки (`cwnd`).

Проблема в контексте BBR. На этом фоне получили развитие подходы, основанные не на потерях, а на моделировании физических ограничений пути: пропускной способности узкого места (`BtlBw`) и задержки распространения (`RTprop`). Алгоритм управления перегрузкой на основе пропускной способности и задержки (Bottleneck Bandwidth and Round-trip propagation time, BBR) является представителем этого направления: он оценивает `BtlBw` и `RTprop`, стремится поддерживать объем данных “в полете” (англ. **inflight** — отправленные пакеты, на которые еще не получено подтверждение) около произведения `BDP = BtlBw × RTprop`, а темп передачи задается с помощью паузы между отправками пакетов в сеть (pacing). Термин **pacing** означает механизм искусственного разнесения отправки пакетов во времени, чтобы они поступали в сеть равномерно, а не пачками. Такой подход позволяет одновременно добиваться пропускной способности транспортного потока, близкой к максимальной пропускной способности соединения, и удерживать задержки на максимально возможном низком уровне, что подтверждается опытом крупномасштабного применения в реальных сервисах и на разнообразных типах

сетей, включая мобильные. BBR был разработан в компании Google и используется для управления трафиком из сервисов компании [4].

Искажение сигналов обратной связи. Применение многопутевой передачи и особенно пакетной балансировки усложняет задачу управления перегрузкой. Транспортный протокол перестает наблюдать “один путь” с единственными характеристиками задержки и пропускной способности. Вместо этого он получает обратную связь от нескольких путей, которые могут различаться по задержкам и доступной пропускной способности. Даже если изначально пути близки по параметрам, в реальной сети неизбежны временные асимметрии: изменения маршрута, сбои, увеличения очереди в буфере и переполнения буфера, приводящие к увеличению задержки или потере части пакетов на одном из путей. В таких условиях пакеты могут приходить не по порядку, что порождает дублирующие ACK и другие аномальные наблюдения, которые TCP склонен трактовать как перегрузку. Это приводит к неоправданному снижению скорости передачи — иногда до состояния, которое можно описать как коллапс пропускной способности при наличии даже небольшой доли перегруженных путей из общего числа путей, между которыми реализована балансировка пакетов [5].

Существующие подходы и их ограничения. Проблема переупорядочивания пакетов рассмотрена в [2]: в этой работе введено понятие **устойчивость к неперегрузочным событиям** (Non-Congestion Robustness) — отправитель не снижает скорость отправки при получении дублирующих ACK, вызванных переупорядочиванием пакетов, а не перегрузкой. Переупорядочивание пакетов, рассмотренное в работе, не сохраняется на протяжении всей передачи. Однако устойчивости только к переупорядочиванию пакетов часто недостаточно. Современные алгоритмы управления перегрузкой используют разные сигналы, например задержку (delay-based). В условиях пакетной балансировки эти сигналы начинают поступать как смесь измерений по разным путям и могут содержать выбросы из-за перегруженного или более длинного пути. Показано, что для широкого класса алгоритмов, использующих AIMD-подобные принципы, при наличии перегруженного пути с вероятностью попадания пакета на перегруженный путь q скорость потока может уменьшаться по закону порядка $\theta(1/\sqrt{q})$. Причем это проявляется в различных алгоритмах управления перегрузкой и сохраняется даже у вариантов, устойчивых к переупорядочиванию [5].

Таким образом, актуальность темы данной выпускной квалификационной работы определяется следующими обстоятельствами:

1. распространенностью и практической значимостью алгоритмов управления перегрузкой, ориентированных на пропускную способность и задержку, таких как BBR, позволяющих приблизиться к высокой скорости соединения при малых очередях [4]

2. растущей ролью пакетной балансировки и других форм многопутевой доставки, которые могут повышать использование доступной суммарной емкости, но провоцируют переупорядочение и делают транспортные измерения смешанными по нескольким путям [1]

1.2. Цель и задачи работы

Цель.

Повышение скорости транспортного соединения за счет алгоритма управления перегрузкой BBR, адаптированного к применению в условиях пакетной балансировки.

Дано.

1. Конечное, упорядоченное множество пакетов, отправленных до настоящего момента $P = \{p_1, p_2, ..., p_n\}$
2. Максимальный размер сегмента MSS.
3. Соответствующее P упорядоченное множество $\mathrm{RTT} = \{\mathrm{RTT}_1, \mathrm{RTT}_2, ..., \mathrm{RTT}_k\}$, где $k \leq n$, так как не на все пакеты могло прийти подтверждение и измерен RTT.
4. d - число последовательно полученных дублирующих ACK для одного и того же неподтвержденного пакета.
5. Оценка пропускной способности узкого места в настоящий момент BtlBw.
6. Оценка минимальной задержки распространения в настоящий момент RTprop.

Требуется.

Построить функцию управления скоростью соединения:

$$S_{\text{new}} = G(S_{\text{BBR}}, \mathrm{RTT}, d, \mathrm{BtlBw}, \mathrm{RTprop}),$$

где:

- S_{BBR} — скорость, предписываемая стандартным алгоритмом BBR,
- RTT — вектор измеренных значений кругового времени,
- d — число последовательных дублирующих ACK,
- BtlBw — оценка пропускной способности узкого места,
- RTprop — оценка минимальной задержки распространения,
- $G(\cdot)$ — искомая функция модификации BBR,
- S_{new} — итоговая скорость передачи.

Цель модификации:

$$S_{\text{new}} \geq S_{\text{BBR}}$$

Задачи:

1. Провести обзор версий BBR.
2. Провести обзор существующих алгоритмов управления перегрузок, адаптированных к применению в условиях пакетной балансировки.
3. Разработать модификацию BBR, направленную на повышение производительности в условиях пакетной балансировки.
4. Провести исследование скорости транспортного потока, управляемого разработанным алгоритмом, в условиях пакетной балансировки

1.3. Структура работы

Данная выпускная квалификационная работа состоит из введения, четырех глав, заключения и списка литературы. В 2 главе проводится обзор версий алгоритма BBR (BBRv1, BBRv2, BBRv3) и анализируются существующие подходы к обеспечению устойчивости алгоритмов управления перегрузкой к переупорядочиванию пакетов (TCP-NCR, Lazy TCP, MSwift). На основе проведенного анализа обосновывается выбор BBRv1 в качестве базы для модификации. В 3 главе на основе выявленных проблем формулируется гипотеза о причинах снижения эффективности BBR при пакетной балансировке и предлагается модификация алгоритмая. В четвертой главе описывается экспериментальное исследование разработанной модификации. Представляется топология сети, параметры соединений и стратегия балансировки, используемые в симуляторе NS-3. Подробно описываются три серии экспериментов: с симметричными путями для базового BBR, с асимметричными путями для базового BBR и с асимметричными путями для модифицированного BBR. Анализируются полученные результаты, формулируются выводы о достигнутом повышении полезной пропускной способности и об ограничениях предложенного решения. В заключении подводятся итоги выполненной работы, обобщаются полученные результаты, отмечается степень достижения поставленной цели и намечаются направления для дальнейших исследований.

2. Обзор литературы

Цель данного обзора состоит в выборе версии алгоритма BBR, наиболее подходящей для использования в условиях пакетной балансировки, а также в определении существующих подходов к обеспечению устойчивости алгоритмов управления перегрузкой к переупорядочиванию пакетов и асимметрии путей.

2.1. Обзор версий BBR

Цель данного обзора состоит в том, чтобы определить, какая из существующих версий BBR подходит для данной работы. Будут рассмотрены только официальные версии, представленные компанией Google, поскольку именно эти версии являются эталонными реализациями, прошли широкое тестирование в реальных сетях и используются в коммерческих продуктах компании. Практическая часть работы будет проводиться в симуляторе сети NS-3 [6], обоснование выбора этого инструмента представлено в главе 4.

Сравнение версий далее будет проводиться по следующим критериям:

- **отсутствие реакции на потерю пакетов** — этот критерий является критически важным, поскольку переупорядочивание пакетов при пакетной балансировке порождает ложные сигналы потери. Если алгоритм снижает скорость при обнаружении потери, он будет ошибочно реагировать на переупорядочивание, что приведет к деградации производительности.
- **наличие реализации в симуляторе NS-3** — этот критерий обусловлен необходимостью проведения экспериментов.

2.1.1. BBRv1

Алгоритм BBR (Bottleneck Bandwidth and Round-trip propagation time) относится к классу model-based алгоритмов управления перегрузкой: его работа основана не на прямой реакции на потери пакетов, а на построении явной модели пути передачи. В основе BBR лежит идея, восходящая к результатам Леонарда Клейнрока [7]: скорость отправки пакетов в сеть должна обеспечивать максимально возможную пропускную способность при минимально необходимом уровне очередей в сети.

В рамках этой модели BBR непрерывно оценивает два ключевых параметра пути:

- минимальную задержку распространения `RTprop` (Round-Trip propagation time), приближаемую наблюдаемым минимумом времени обхода пакета туда и обратно. Заметим, что `RTprop` — это характеристика пути, равнвя сумме задержки распространения и задержки пакетизации, и она не может быть меньше некоторого базового значения. Любое превышение этого базового значения говорит о наличии очередей.
- пропускную способность узкого места `BtlBw` (Bottleneck Bandwidth), оцениваемую по скорости доставки данных получателю. Узкое место — это участок сети с наименьшей пропускной способностью на пути следования пакетов. Именно оно ограничивает максимальную скорость передачи данных между отправителем и получателем.

Согласно [7] оптимальный объем данных в сети, то есть суммарный размер пакетов, на которые не было получено подтверждение, определяется произведением пропускной способности на задержку распространения (bandwidth-delay product, `BDP`):

$$\mathrm{BDP} = \mathrm{BtlBw} \cdot \mathrm{RTprop}$$

Это произведение имеет простой смысл: оно равно максимальному количеству данных, которое может находиться в сети (в каналах и буферах) между отправителем и получателем при полном использовании пропускной способности узкого места. Если отправитель будет поддерживать объем неподтвержденных данных меньше `BDP`, канал будет недогружен. Если объем неподтвержденных данных превышает `BDP`, избыточные данные будут накапливаться в буферах, увеличивая задержки, но не увеличивая пропускную способность. Таким образом, `BDP` задает естественную целевую оценку числа данных “в полете” (`inflight`), необходимого для насыщения канала без формирования избыточной очереди.

Соответственно, BBR управляет соединением с помощью регулирования темпа отправки пакетов (`pacing`). Этот механизм обеспечивает равномерный поток пакетов в сеть, предотвращая образование микро-всплесков, которые могут привести к переполнению буферов даже при среднем темпе ниже пропускной способности канала.

Темп передачи выбирается пропорционально текущей оценке пропускной способности узкого места:

$$\mathrm{pacing_rate} = \mathrm{pacing_gain} \cdot \mathrm{BtlBw}$$

Здесь `pacing_gain` — коэффициент, который определяет, должен ли отправитель ускорить передачу, поддерживать стационарный режим или временно снижать интенсивность для опустошения очереди. Работу BBR можно представить как конечный автомат состояний. Коэффициент `pacing_gain` может быть различным в различных состояниях, схема автомата изображена на рисунке 1.

Механизм оценки BtlBw. Оценка `BtlBw` формируется по наблюдаемой скорости доставки данных. В BBR она вычисляется как максимум скорости доставки (`delivery_rate`) на скользящем интервале длиной порядка десяти раундов передачи. Скорость доставки — это количество данных, подтвержденное за единицу времени. Если отправитель получает подтверждения на пакеты, он может вычислить, с какой скоростью эти пакеты были доставлены получателю. Формально, скорость доставки определяется как:

$$\mathrm{delivery_rate} = \frac{\mathrm{data_delivered}}{\mathrm{time_elapsed}}$$

где `data_delivered` — объем данных, подтвержденных за некоторый интервал времени, а `time_elapsed` — длина этого интервала. Затем BBR берет максимум этой величины на скользящем окне:

$$\mathrm{BtlBw} = \max_{t \in W} \mathrm{delivery_rate}(t), \quad |W| \approx 10 \cdot \mathrm{RTprop}$$

Тем самым BBR стремится выделить устойчивую верхнюю оценку доступной пропускной способности, подавляя кратковременные просадки, которые могут быть вызваны, например, временными помехами или конкуренцией с другими потоками.

Механизм оценки RTprop. Оценка `RTprop`, напротив, выбирается как минимальный RTT на достаточно длинном временном окне, поскольку именно минимальное наблюдаемое значение наилучшим образом соответствует задержке распространения и пакетизации без влияния очередей:

$$\mathrm{RTprop} = \min_{t \in W_r} \mathrm{RTT}(t)$$

,

где $W_r = 10$с

Таким образом, базовая логика BBR может быть интерпретирована как поддержание условия

$$\mathrm{inflight} \approx \mathrm{BDP}$$

то есть такого режима, при котором соединение заполняет канал, но по возможности не создает устойчивой очереди в узком месте.

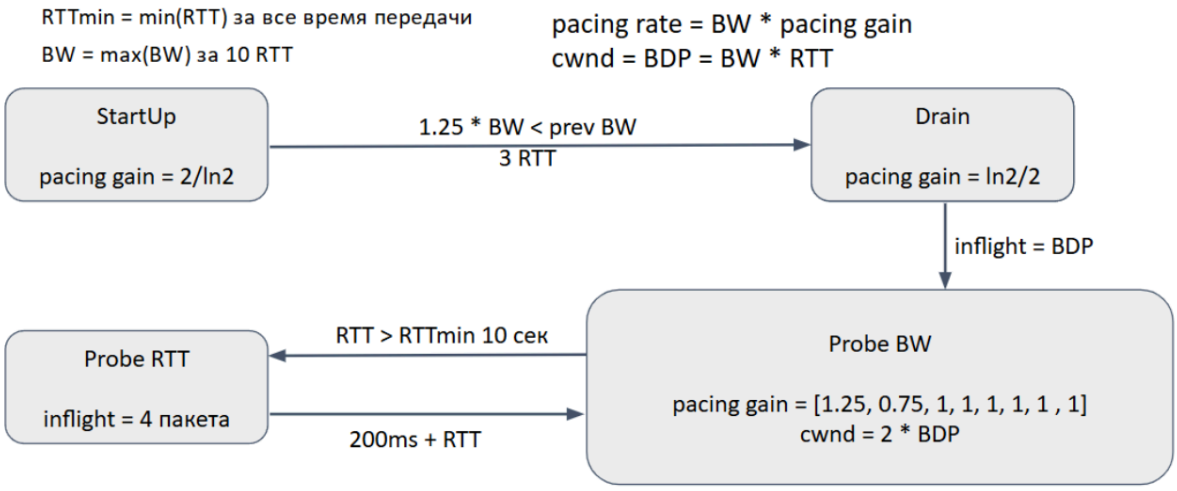

Рисунок 1. Схема автомата BBR.

Функционирование BBR удобно описывать как работу конечного автомата из четырех основных состояний [8].

Состояние StartUp. На начальном этапе отправитель стремится быстро приблизиться к доступной пропускной способности пути. Для этого используется повышающий коэффициент темпа передачи

$$\mathrm{pacing_gain} = \frac{2}{\ln 2} \approx 2.89$$

что соответствует экспоненциальному наращиванию интенсивности отправки до тех пор, пока оценки скорости доставки продолжают заметно расти. Выбор именно такого значения `pacing_gain` гарантирует, что за каждый раунд передачи окно перегрузки удваивается, аналогично фазе медленного старта в классическом TCP, но с учетом того, что BBR использует pacing для сглаживания отправки. Переход из состояния StartUp выполняется тогда, когда прирост оценки `BtlBw` перестает быть существенным в течение нескольких последовательных раундов передачи; в реализации BBR это интерпретируется как достижение насыщения узкого места. Конкретно, переход происходит, если за три раунда передачи оценка `BtlBw` выросла менее чем на 25%.

Состояние Drain. После агрессивного заполнения канала, характерного для StartUp, в сети обычно формируется избыточная очередь. Поэтому на этапе Drain отправитель временно снижает темп передачи, чтобы удалить накопившиеся данные из буфера и вернуть соединение к режиму, близкому к целевому `BDP`. Для этого используется коэффициент

$$\text{pacing_gain} = \frac{\ln 2}{2} \approx 0.35$$

Это значение является обратным по отношению к `pacing_gain` в состоянии StartUp. Если в StartUp отправитель увеличивал темп в 2.89 раз, то в Drain он уменьшает его примерно в те же 2.89 раз, но поскольку объем данных в сети не может уменьшиться мгновенно, это приводит к быстрому опустошению очереди. Переход к следующему состоянию происходит, когда объем данных в сети становится не больше целевого значения, то есть выполняется условие

$$\text{inflight} \leq \text{BDP}$$

Состояние ProbeBW. Это основное рабочее состояние алгоритма, в котором соединение проводит большую часть времени. Его задача состоит в периодическом зондировании доступной пропускной способности: отправитель чередует фазы умеренного ускорения, умеренного замедления и нейтральной передачи. В классическом варианте BBR используется циклическая последовательность коэффициентов

$$(1.25, 0.75, 1, 1, 1, 1, 1, 1)$$

с переключением между фазами примерно один раз за RTT. Фаза с коэффициентом 1.25 (ускорение) позволяет проверить, не увеличилась ли доступная пропускная способность — если да, то оценка `BtlBw` обновится в сторону увеличения, и соединение сможет занять освободившиеся ресурсы. Фаза с коэффициентом 0.75 (замедление) способствует уменьшению возможной очереди, которая могла накопиться во время фазы ускорения. Нейтральные фазы с коэффициентом 1 позволяют соединению работать в стационарном режиме. В этом состоянии окно перегрузки обычно выбирается с запасом относительно `BDP` (`cwnd = 2 dot BDP`), чтобы pacing оставался основным механизмом управления скоростью, а `cwnd` не ограничивал передачу преждевременно. Этот запас объясняется тем, что в фазе ускорения темп отправки превышает `BtlBw`, и `cwnd` должно быть достаточно большим, чтобы не ограничивать отправителя.

Состояние ProbeRTT. Поскольку минимальная задержка распространения может со временем устаревать из-за постоянного присутствия очереди (например, если в сети долгое время есть другие потоки, создающие очередь на узком месте), BBR периодически переходит в режим переоценки `RTprop`. Для этого объем данных в сети искусственно уменьшается, а окно перегрузки сокращается до малого значения (до четырех сегментов), чтобы дать очередям возможность полностью опустеть. После короткого интервала наблюдения (обычно 200 мс), достаточного для измерения нового минимального RTT, автомат возвращается в состояние ProbeBW [4]. Состояние ProbeRTT входит в режим, если прошло более 10 секунд с момента последнего обновления `RTprop`.

Реакция на потерю пакетов. BBRv1 не изменяет свое поведение при потере пакетов. Потерянные пакеты повторно отправляются механизмом `Fast Retransmit`, однако алгоритм управления перегрузкой не снижает скорость передачи. Это свойство делает BBRv1 потенциально устойчивым к переупорядочиванию пакетов: ложные сигналы потери, возникающие из-за переупорядочивания, не приводят к уменьшению темпа отправки. Однако, как будет показано в главе 4, даже при отсутствии реакции на потерю проблема полного использования пропускной способности многопутевого соединения остается нерешенной.

2.1.2. BBRv2

Вторая версия BBR была представлена в 2019 году [8]. Главное изменение алгоритма — добавление реакции на потерю пакетов для борьбы с чрезмерным заполнением буферов. Опыт эксплуатации BBRv1 в масштабах интернета показал, что при определенных условиях алгоритм может заполнять буферы слишком агрессивнор. BBRv2 пытается решить эту проблему, вводя механизм, снижающий скорость при обнаружении потерь.

В BBRv2 введено условие `loss`, которое выполняется, если за время, равное одному RTT, потеряно не менее 8 пакетов или доля потерянных пакетов составляет 0.02 и более. При выполнении этого условия алгоритм снижает целевую величину `inflight_hi` (верхняя граница объема данных в сети) до значения `BDP`. Таким образом, BBRv2 становится более “дружелюбным” к другим потокам в условиях больших буферов, но приобретает чувствительность к потерям, в том числе к ложным потерям, вызванным переупорядочиванием пакетов.

Расширенная структура состояний. Автомат BBRv2 сохранил структуру автомата BBRv1, за исключением нескольких изменений: в состоянии ProbeRTT размер окна установлен равным `BDP/2` (вместо четырех сегментов в BBRv1), что позволяет реже входить в это состояние и менее резко снижать скорость. Кроме того, состояние ProbeBW претерпело существенные изменения: оно разбито на четыре подсостояния: ProbeDown, ProbeCruise, ProbeRefill и ProbeUp. Эта детализация позволяет более тонко управлять процессом зондирования пропускной способности с учетом наличия потерь.

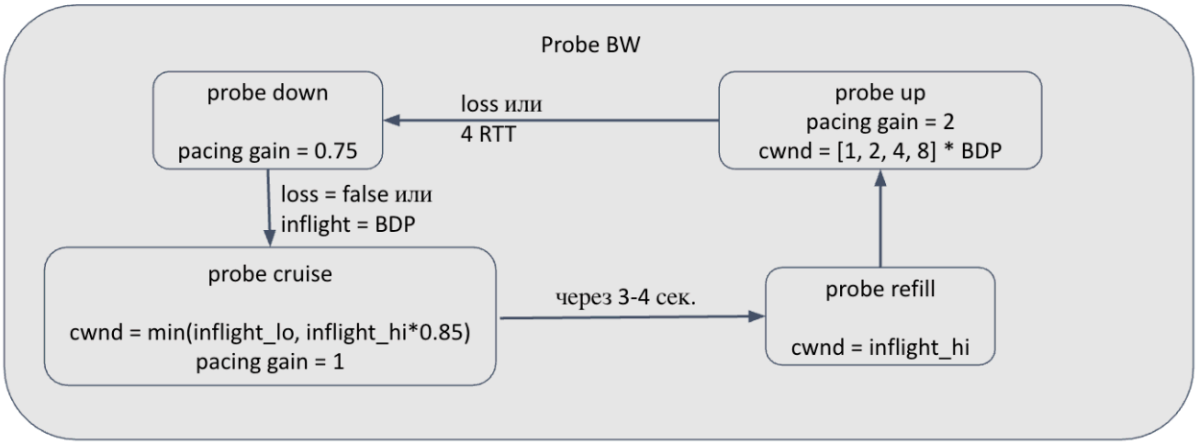

Рисунок 2. Подсостояния состояния ProbeBW автомата BBRv2.

Подсостояние ProbeDown. Это подсостояние активируется при переходе в ProbeBW, если выполняется условие `loss` (обнаружены потери). Коэффициент увеличения темпа установлен равным 0.75. Это подсостояние нужно для того, чтобы избавиться от потерь — отправитель снижает темп, давая очередям возможность опустеть. Когда условие `loss` перестает выполняться (т.е. потери прекращаются), автомат переходит в следующее подсостояние — ProbeCruise.

Подсостояние ProbeCruise. В этом подсостоянии коэффициент увеличения темпа равен 1. Отправитель работает в стационарном режиме, поддерживая достигнутый уровень скорости. Через 3–4 секунды (это время достаточно для того, чтобы убедиться, что потери не возвращаются) автомат переходит в подсостояние ProbeRefill.

Подсостояние ProbeRefill. В этом состоянии отправитель немного увеличивает поток пакетов, чтобы подготовиться к следующему подсостоянию. Цель — постепенно “наполнить” окно до уровня, необходимого для эффективного зондирования. Через один RTT автомат переходит в подсостояние ProbeUp.

Подсостояние ProbeUp. Коэффициент увеличения темпа равен 2, окно перегрузки последовательно принимает значения (1, 2, 4, 8) на каждом интервале, равном RTT. Это подсостояние предназначено для занятия освободившихся ресурсов сети, если таковые имеются. Если во время этого процесса выполняется условие `loss` (т.е. начинаются потери) или проходит время, равное 4 RTT, автомат возвращается в подсостояние ProbeDown.

Наличие явной реакции на потерю пакетов в BBRv2 означает, что при переупорядочивании, вызванном пакетной балансировкой, алгоритм может ошибочно снижать скорость передачи, интерпретируя дублирующие ACK как признаки перегрузки.

2.1.3. BBRv3

Третья версия BBR была представлена в 2024 году [9]. Автомат алгоритма не изменился по сравнению с BBRv2, но были изменены коэффициенты, управляющие потоком. Эти изменения направлены на повышение агрессивности алгоритма при захвате доступной пропускной способности. В состоянии StartUp константа ускорения темпа равна 2.77 (вместо 2.89 в BBRv1 и BBRv2), в подсостоянии ProbeUp состояния ProbeBW она равна

2.25, а в подсостоянии ProbeDown — 0.9. Таким образом, большую часть времени передачи поток пакетов, управляемый BBRv3, более агрессивно занимает ресурсы сети, что может давать преимущество в конкурентной среде, но также может усиливать проблемы с переупорядочиванием.

Анализ параметров BBRv3. Изменение коэффициента в StartUp с 2.89 на 2.77 означает небольшое снижение начальной агрессивности, что может уменьшить риск создания избыточной очереди на этапе первичного насыщения канала. Однако увеличение коэффициента в ProbeUp с 1.25 (в BBRv2) до 2.25 означает значительно более агрессивное зондирование пропускной способности в рабочем режиме. Это может привести к более быстрому захвату освободившихся ресурсов, но также может создавать значительные колебания очереди. Снижение коэффициента в ProbeDown с 0.75 до 0.9 означает менее глубокое уменьшение скорости при обнаружении потерь, что может помочь быстрее восстанавливаться после ложных срабатываний, но может быть недостаточным для устранения реальной перегрузки.

Параметр	BBRv1	BBRv2	BBRv3
Коэффициент StartUp	$\frac{2}{\ln 2}$	$\frac{2}{\ln 2}$	2.77
Коэффициент ProbeUp	1.25	2	2.25
Коэффициент ProbeDown	0.75	0.75	0.9

Таблица 1. Значения коэффициентов темпа в трех версиях BBR. В таблице приведены значения paching_gain для ключевых состояний алгоритма: StartUp (начальное зондирование), ProbeUp (активная фаза зондирования пропускной способности) и ProbeDown (фаза снижения после обнаружения потерь). Видно, что BBRv3 имеет промежуточные значения между BBRv1 и BBRv2, а также уникальные значения для некоторых состояний.

Как и BBRv2, третья версия реагирует на потерю пакетов, что делает ее чувствительной к переупорядочиванию. Кроме того, реализация BBRv3 в симуляторе NS-3 отсутствует, что ограничивает возможность ее экспериментального исследования в рамках данной работы. На момент написания работы (2025-2026 годы) активная разработка и внедрение BBRv3 продолжаются, но стабильная реализация для NS-3 еще не выпущена.

2.1.4. Выбор версии BBR для дальнейшего исследования

	BBRv1	BBRv2	BBRv3
Отсутствие реакции на потерю пакетов	Да	Нет	Нет
Реализация в NS-3	Да	Да	Нет

Таблица 2. Сравнение версий BBR

BBRv1 не реагирует на потерю пакетов и реализован в NS-3. BBRv2 и BBRv3 содержат механизмы снижения скорости при обнаружении потерь, что может

приводить к деградации производительности при пакетной балансировке, поскольку переупорядочивание пакетов порождает ложные сигналы потерь. Кроме того, реализация BBRv3 в NS-3 отсутствует, что делает невозможным его экспериментальное исследование на данный момент. Таким образом, для дальнейшего исследования в рамках данной работы выбрана первая версия BBR.

2.2. Обзор существующих решений для работы алгоритмов управления перегрузкой в условиях пакетной балансировки

Цель данного обзора состоит в том, чтобы определить, какие из существующих подходов к обеспечению устойчивости TCP к пакетной балансировке могут быть использованы как основа для модификации BBRv1. Для этого рассматриваются не только сами механизмы сохранения скорости передачи при переупорядочивании пакетов и асимметрии путей, но и степень их совместимости с моделью управления перегрузкой, лежащей в основе BBR. Важно понимать, что BBR принципиально отличается от классического TCP: он не использует потерю как основной сигнал перегрузки, не полагается на AIMD-подобное управление окном (Additive Increase Multiplicative Decrease, аддитивное увеличение и мультипликативное уменьшение скорости отправки) и не снижает скорость при получении дублирующих ACK. Поэтому многие решения, эффективные для классического TCP, могут быть неприменимы к BBR.

Сравнение решений далее проводится по следующим критериям:

- **Сохранение скорости передачи при переупорядочивании пакетов.** Этот критерий показывает, способен ли алгоритм не интерпретировать обычное для пакетной балансировки нарушение порядка доставки как сигнал потери. Он важен, поскольку ложные срабатывания `Fast Retransmit` приводят к избыточным повторным передачам и снижают полезную пропускную способность.
- **Сохранение скорости передачи при асимметрии путей.** Здесь оценивается, насколько хорошо подход справляется с ситуацией, когда лишь часть маршрутов становится хуже по задержке или пропускной способности на короткий период. Этот критерий важен для нашей работы, потому что в реальных условиях пакетной балансировки пути редко остаются полностью постоянными — задержки могут меняться из-за изменений нагрузки, маршрутизации или внешних факторов.
- **Независимость от стратегии балансировки и количества маршрутов.** В задаче нет ограничений на стратегию балансировки в сети. Алгоритм не должен полагаться ни на какие предположения о том, как происходит балансировка пакетов между маршрутами, сколько существует маршрутов для балансировки и происходит ли балансировка вообще. Это требование особенно важно, поскольку реальные сети могут использовать различные стратегии балансировки (round-robin, взвешенное распределение, хэширование по потокам и т.д.), и алгоритм управления перегрузкой должен работать эффективно при любом из этих подходов.
- **Совместимость с моделью BBR.** Под этим понимается, можно ли встроить рассматриваемый механизм в BBR без отказа от его базовых принципов — оценок

BtlBw и RTprop, управления скоростью отправки пакетов через pacing_rate и model-based логики в целом.

2.2.1. TCP-NCR

TCP-NCR (Non-Congestion Robustness) был предложен как модификация классического TCP, в которой запуск Fast Retransmit и последующее уменьшение окна перегрузки тесно связаны с получением серии дублирующих ACK. Основная идея метода состоит в том, чтобы сделать порог срабатывания не фиксированным (3 дублирующих ACK), а зависящим от текущего состояния соединения: чем больше данных находится «в полете», тем больший уровень переупорядочивания допускается без немедленного объявления потери [2].

Обоснование динамического порога. В классическом TCP порог в 3 дублирующих ACK был выбран эмпирически как компромисс между скоростью реакции на реальные потери и устойчивостью к случайным переупорядочиваниям. Однако в современных высокоскоростных сетях с большим объемом данных “в полете” даже небольшой процент переупорядоченных пакетов может генерировать большое количество дублирующих ACK.

Порог задается выражением

$$\mathrm{DupThresh} = \max\left(\mathrm{LT_F} \cdot \frac{\mathrm{InFlightSize}}{\mathrm{MSS}}, 3\right),$$

где InFlightSize — текущий объем неподтвержденных данных, MSS — максимальный размер сегмента, а LT_F — параметр настройки алгоритма. Знаменатель MSS используется для перевода объема данных в количество пакетов, поскольку дублирующие ACK подсчитываются поштучно для пакетов. Параметр LT_F (Loss Tolerance Factor) определяет долю переупорядоченных пакетов, которую алгоритм готов терпеть без реакции. Авторы TCP-NCR рекомендуют значение LT_F = 0.1, что означает терпимость к переупорядочиванию до 10% от объема данных в полете.

TCP-NCR сохраняет скорость передачи при переупорядочивании пакетов, так как увеличивает порог срабатывания и, следовательно, снижает количество ложных срабатываний Fast Retransmit. Однако его работа зависит от настройки параметра LT_F. В целом предложенный подход полезен для loss-based TCP, поскольку в нем дублирующие ACK являются одним из центральных сигналов, определяющих и факт потери, и момент снижения cwnd. Однако для BBR этот механизм концептуально не подходит. BBRv1 не снижает скорость передачи при получении дублирующих ACK. Его основное управление строится вокруг модели пути, включающей оценки BtlBw и RTprop, а темп передачи задается через pacing_rate, а не через cwnd. Динамическая настройка DupThresh может быть полезна для уменьшения числа ложных повторных передач, но сама по себе не решает проблему искажения измерений. Тем не менее, сама идея динамического порога является ценной и будет адаптирована для BBR в главе 3.

2.2.2. Lazy TCP (LTCP)

Lazy TCP также разрабатывался для сценария packet spraying, но, как и TCP-NCR, остается привязанным к классической TCP-логике управления через потери и окно перегрузки. В работе [10] предложены два основных механизма.

Первый механизм — **lazy trigger**: отправитель не реагирует немедленно на серию дублирующих ACK, а откладывает запуск восстановления, чтобы дать переупорядоченным пакетам возможность прийти без ложного уменьшения скорости. Вместо того чтобы запускать Fast Retransmit сразу при получении третьего дублирующего ACK, LTCP запускает таймер небольшой длительности (порядка нескольких миллисекунд). Если за это время ожидаемый пакет прибывает (т.е. переупорядочивание разрешается само собой), то ложное срабатывание предотвращается. Если же таймер истекает, то пакет действительно считается потерянным.

Второй механизм — **stubborn window**: алгоритм различает **дискретные** и **последовательные** потери. Для работы механизма используется опция `SACK` (Selective Acknowledgement) — при включении этой опции ACK пакеты содержат информацию о номерах пакетов, которые были получены вместо ожидаемого. По сути, SACK позволяет получателю сообщить отправителю не только номер следующего ожидаемого пакета, но и список уже полученных пакетов, что дает отправителю полную картину о том, какие пакеты доставлены, а какие нет. Если по информации `SACK` видно, что потери единичны и не образуют длинную непрерывную последовательность, LTCP трактует их как следствие переупорядочивания пакетов из-за отдельного пути и не уменьшает `cwnd`; если же потери выглядят как серия подряд идущих сегментов (например, пять и более пакетов подряд не подтверждены), они интерпретируются как признак перегрузки, и окно уменьшается. Критерий «последовательности» задается порогом `k`, который трактуется как ожидаемое количество соседних пакетов, попавших на один перегруженный путь при балансировке.

Такой алгоритм полезен для избежания излишних снижений скорости при реальной перегрузке сети. Однако BBRv1 не использует дублирующие ACK как сигнал о перегрузке, следовательно, механизм lazy trigger для него избыточен — BBRv1 и так не снижает скорость при дублирующих ACK. Механизм stubborn window также не применим, поскольку BBRv1 не имеет окна перегрузки в классическом понимании. Кроме того, в алгоритме используется параметр `k`, который нужно настраивать исходя из предположений о стратегии балансировки в сети. В задаче не делается таких предположений. Количество соседних пакетов, попавших на один и тот же путь при балансировке, может значительно меняться на протяжении всей передачи. Вследствие этого рассмотренный алгоритм не подходит по критериям обзора.

2.2.3. MSwift

В работе [5] показано, что снижение скорости передачи при асимметрии путей связано не только с переупорядочиванием пакетов, но и с искажением измерений скорости доставки. Это более глубокая проблема: даже если полностью устранить ложные срабатывания Fast Retransmit, алгоритмы управления перегрузкой все равно получают искаженные данные о RTT и скорости доставки, поскольку эти измерения получаются из нескольких путей с разными характеристиками. В связи с этим в MSwift рассматривается не только проблема

дублирующих ACK, но и общая задача обработки сигналов перегрузки при пакетной балансировке.

В MSwift наряду с механизмами LTCP используется медианная агрегация задержки: вместо реакции на отдельное измерение RTT алгоритм вычисляет медиану по окну последних наблюдений и затем принимает решение об уменьшении окна, если медианная задержка превышает заданный порог.

BBRv1 не реагирует на кратковременные повышения задержки отдельных пакетов. В нем измеряется минимальная задержка доставки, и это измерение используется при управлении скоростью потока. Периодическое увеличение задержки, вызванное пакетной балансировкой, игнорируется, поскольку BBR сохраняет только минимальные значения. Аналогичная проблема возникает и для оценки пропускной способности. В BBR параметр `BtlBw` строится как максимум наблюдаемой `delivery_rate` на скользящем окне. Кратковременные понижения этого измерения допустимы — максимум их отфильтрует. Следовательно, MSwift не подходит по критериям обзора, хотя его подход к робастной обработке измерений заслуживает внимания для будущих исследований.

2.2.4. Вывод

	TCP-NCR	LTCP	MSwift
Сохранение скорости передачи при переупорядочивании пакетов	Да	Да	Да
Сохранение скорости передачи при ассиметрии каналов	Нет	Нет	Да
Независимость от стратегии балансировки	Да	Нет	Нет
Применимость к BBR	Да	Нет	Нет

Таблица 3. Сравнение подходов TCP-NCR, LTCP и MSwift.

Рассмотренные подходы решают разные частные проблемы пакетной балансировки, однако ни один из них не может быть непосредственно использован как модификация BBRv1. TCP-NCR и LTCP ориентированы на классический TCP, в котором ключевыми управляющими событиями являются дублирующие ACK, `Fast Retransmit` и изменение `cwnd` после потерь. Для BBR такие механизмы вторичны: его поведение определяется моделью пути, оценками `BtlBw` и `RTprop`, а также управлением скоростью отправки пакетов через `pacing_rate`.

MSwift поднимается на более общий уровень и показывает, что при многопутевой доставке требуется обработка сигналов, учитывающая искажения измерений скорости доставки. Тем не менее его конкретная реализация также не подходит для BBR, поскольку основана на медианной оценке RTT и пороговом уменьшении `cwnd`, тогда как BBR нуждается в оценках другого типа: минимальной задержки распространения и верхней оценки пропускной способности.

Следовательно, направление модификации BBR должно опираться не на прямое заимствование TCP-NCR, LTCP или MSwift, а на сохранение базовой модели BBR при разработке специальных, совместимых с ней методов интерпретации многопутевых измерений. В качестве базовой идеи для модификации выбрано повышение порога `DupThresh`, предложенного в TCP-NCR, однако формула расчета порога адаптирована для BBR с учетом того , что BBR не снижает скорость при дублирующих ACK, а только страдает от избыточных повторных передач.

3. Адаптация BBR к условиям пакетной балансировки

Основная причина уменьшения пропускной способности BBR при пакетной балансировке связана со снижением полезной скорости из-за повторных передач, вызванных переупорядочиванием пакетов при асимметрии задержек. При этом ключевую роль играет стандартный механизм `Fast Retransmit`, который при получении трех дубликатных ACK инициирует повторную отправку предполагаемого потерянным пакета, даже если фактической потери не было, а пакет лишь задержался на более длинном пути. Причем пропускная способность соединения сохранится, ведь BBR не реагирует на сигналы о потере пакета. Таким образом, в сеть будут отправляться излишние дубликаты пакетов, которые займут часть пропускной способности сети. Из-за этого полезная пропускная способность транспортного соединения (goodput) уменьшится.

Под **полезной пропускной способностью** (goodput) понимается скорость доставки полезных данных приложения, то есть общий объем данных за исключением повторных передач, успешно доставленных получателю, деленный на время передачи. В условиях пакетной балансировки goodput может быть существенно ниже пропускной способности канала из-за того, что часть полосы занимают повторно переданные пакеты.

В условиях асимметрии задержек пакет, попавший на канал с большей задержкой, прибудет к получателю позже пакетов, отправленных по быстрому пути, на величину разности задержек на путях. За время этой задержки отправитель успевает получить от быстрых пакетов несколько дублирующих ACK и запускает повторную передачу. Объем данных, который может находиться “в полете” без получения подтверждения по медленному пути, ограничен произведением пропускной способности узкого места на разность задержек. Пусть балансировка происходит между двумя маршрутами, имеющими задержки d_f и d_s ($d_s > d_f$), максимальное число пакетов, которые могут оказаться переупорядоченными, определяется величиной

$$N_{\max} = \min\left(\left\lceil \frac{\mathrm{BtlBw} \cdot (d_s - d_f)}{\mathrm{MSS}} \right\rceil, \mathrm{cwnd}\right)$$

где `MSS` — размер сегмента. Физический смысл этой формулы: за время разности задержек `d_s - d_f` отправитель успевает отправить `BtlBw * (d_s - d_f)` битов данных. Разделив на размер сегмента `MSS`, получаем количество сегментов, которые могут быть отправлены и подтверждены по быстрому пути, пока пакет, отправленный по медленному пути, еще находится в пути. Именно столько дублирующих ACK может быть сгенерировано получателем, прежде чем придет пакет с медленного пути.

Поскольку d_s не может превышать максимальный RTT, наблюдаемый на соединении, а d_f не может быть меньше `RTprop`, получаем верхнюю оценку:

$$d_s \le \frac{\mathrm{RTT}_{\max}}{2}, d_f \ge \mathrm{RTprop} \Rightarrow N_{\max} \le \frac{(\mathrm{RTT}_{\max} - \mathrm{RTprop}) \cdot \mathrm{BtlBw}}{2 \cdot \mathrm{MSS}}$$

где $\mathrm{RTT}_{\max}$ — максимальный наблюдаемый RTT пакета, который достигается в случае, когда и сам пакет, и его ACK попадают на путь с большей задержкой. Максимум $N_{\max}$

достигается в ситуации, когда один пакет `p` в окне попадает на путь с большей задержкой, а все остальные пакеты — на путь с меньшей задержкой. Это наихудший с точки зрения переупорядочивания сценарий, так как все остальные пакеты будут доставлены быстро и сгенерируют множество дублирующих ACK, в то время как медленный пакет задержится.

Следовательно, фиксированный порог в три `DupACK` оказывается непригодным: он многократно занижает допустимую степень переупорядочивания и провоцирует ложные повторные передачи, которые забивают канал дублирующими данными. При $\mathrm{RTT}_{\max} = 3 \cdot \mathrm{RTprop}$, $N_{\max} = \frac{\mathrm{BDP}}{\mathrm{MSS}}$, то есть количество дублирующих ACK, которые мы можем игнорировать, равно количеству пакетов в BDP. Дальнейшее увеличение порога приведет к тому, что количество неподтвержденных отправленных пакетов превысит $\frac{\mathrm{BDP}}{\mathrm{MSS}}$, в таком случае BBR начнет снижать скорость передачи, до тех пор пока не получит подтвержение на пакет `p`.

В качестве решения предлагается модифицировать алгоритм BBR, заменив фиксированный порог срабатывания `Fast Retransmit` динамическим значением:

$$\mathrm{DupACKThresh} = \max\left(\frac{\mathrm{BDP}}{\mathrm{MSS}}, 3\right)$$

Предлагаемое изменение не затрагивает центральные механизмы BBR — оценку `BtlBw` и `RTprop` и управление скоростью через `pacing_rate`. Также решение не требует настройки параметров, строящейся на предположении о балансировке в сети. При $\mathrm{RTT}_{\max} > 3 \cdot \mathrm{RTprop}$ ложные повторные передачи также возможны, но их количество уменьшится.

4. Экспериментальное исследование работы BBR в условиях пакетной балансировки

Для исследования поведения алгоритма управления перегрузкой в условиях применения пакетной балансировки требуется экспериментальное исследование в среде, позволяющей изолированно изучать указанные в выше эффекты. В качестве основного инструмента выбран симулятор сети NS-3 версии 3.45 (Network Simulator 3)[6]. Этот выбор обусловлен необходимостью детерминированного воспроизведения сценариев с контролируемыми параметрами задержки и пропускной способности на каждом из балансируемых путей, а также наличием корректной реализации BBR [11,12].

4.1. Топология сети

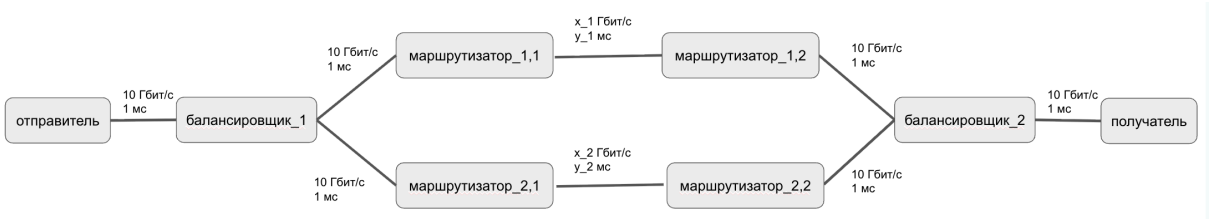

Рисунок 3. Топология сети в эксперименте.

Для экспериментального исследования был выбран сценарий, моделирующий передачу данных между одним отправителем и одним получателем через два последовательных уровня пакетной балансировки. Топология сети, изображенная на рисунке, включает в себя следующие узлы: источник трафика (отправитель), два балансировщика нагрузки (балансировщик_1 и балансировщик_2), два альтернативных маршрута, а также конечный узел-получатель (получатель). Отправитель соединен с первым балансировщиком, который распределяет поступающие пакеты по 2 альтернативным маршрутам. Каждый из этих маршрутов содержит один роутер, после чего все потоки повторно агрегируются вторым балансировщиком и направляются получателю. Количество альтернативных маршрутов = 2. Ограничение обусловлено тем, что в большинстве топологий, использующих балансировку (по потокам или по пакетам), число альтернативных непересекающихся маршрутов не превосходит 2 [13]. Различные настройки параметров соединений между маршрутизаторами позволяют моделировать симметричные и асимметричные параллельные маршруты.

Участок сети	Пропускная способность	Задержка
отправитель - балансировщик$_1$	10 Gbps	1 ms
балансировщик$_1$ - маршрутизатор	10 Gbps	1 ms
маршрутизатор$_{1,1}$ - маршрутизатор$_{1,2}$	x_1 Mbps	y_1 ms
маршрутизатор$_{2,1}$ - маршрутизатор$_{2,2}$	x_2 Mbps	y_2 ms
маршрутизатор - балансировщик$_2$	10 Gbps	1 ms
балансировщик$_2$ - получатель	10 Gbps	1 ms

Таблица 4. Параметры соединений в эксперименте

Параметры всех сетевых соединений приведены в таблице 4. Участок отправитель-балансировщик$_1$ и все последующие участки, характеризуются высокой пропускной способностью (10 Гбит/с) и фиксированной задержкой в 1 мс, что исключает ограничение скорости передачи данных в этих соединениях. Параметры множества альтернативных маршрутов маршрутизатор$_{i,1}$ - маршрутизатор$_{i,2}, i \in [1, 2]$ задают условия эксперимента для каждого маршрута. Для каждого такого канала индивидуально определяются два ключевых показателя: пропускная способность (data_rate) и задержка (delay). Размер очереди для всех каналов установлен удвоенным произведением пропускной способности на задержку ($2 \cdot \text{data_rate} \cdot \text{delay} = 2 \cdot \text{BDP}$), что соответствует оптимальному значению для буферов при использовании алгоритма BBR в транспортном агенте [8].

Для моделирования балансировки, написан класс L3 балансировщик, использующий стратегию round robin при балансировке. При маршрутизации ACK пакетов, также происходит балансировка, причем ACK может пойти не по тому маршруту, по которому шел пакет.

4.2. Экспериментальное исследование скорости передачи данных в условиях симметричных маршрутов при управлении базовым алгоритмом

Первая серия экспериментов проведена для оценки поведения базового алгоритма BBR в идеализированных условиях, когда все параллельные пути между балансировщиками обладают идентичными характеристиками. Симметричная конфигурация (при $N = 2$) позволяет установить базовую линию производительности BBR при пакетной балансировке, исключив влияние асимметрии задержек или пропускной способности. Этот эксперимент важен для понимания того, насколько BBR в принципе способен использовать несколько параллельных путей, даже в самых благоприятных условиях.

В данной серии экспериментов параметры каждого из каналов между маршрутизаторами заданы следующим образом:

- пропускная способность (`data_rate`) — от 50 `Мбит/с` до 1 `Гбит/с`;
- задержка распространения (`delay`) — 10 `мс`;

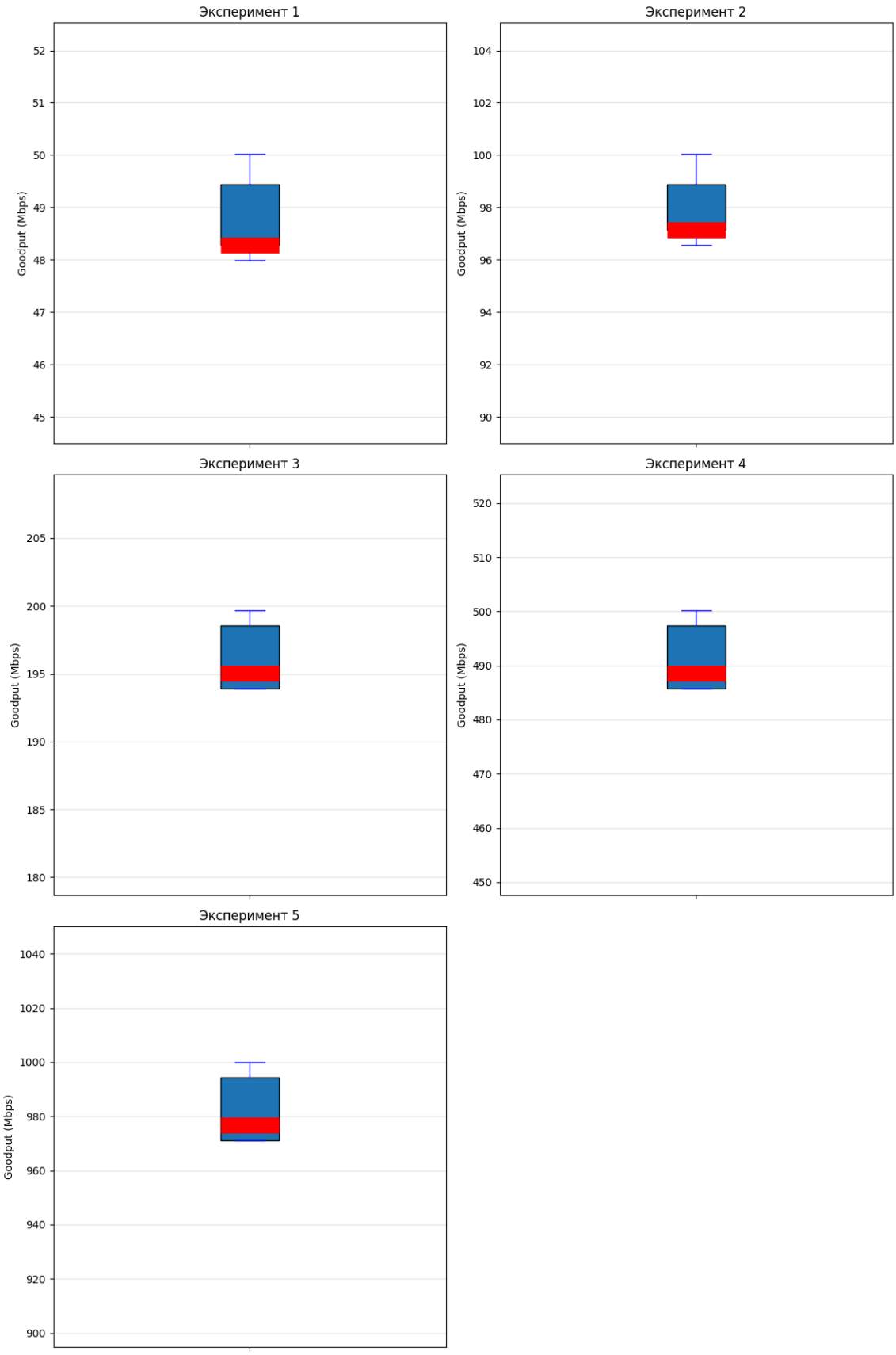

Рисунок 4. Диаграмма размаха полезной пропускной способности BBR во времени при симметричных альтернативных путях с скоростью передачи: 1 - 50 Мбит/с, 2 - 100 Мбит/с, 3 - 200 Мбит/с, 4 - 500 Мбит/с, 5 - 1 Гбит/с.

Результаты эксперимента. В первом эксперименте (Рисунок 3) среднее значение полезной пропускной способности равно 48.2 `Мбит/с`, во втором — 97.1 `Мбит/с`, в третьем — 195 `Мбит/с`, в четвертом — 489 `Мбит/с`, в пятом — 988 `Мбит/с`. Несмотря на наличие двух альтернативных путей в симметричной конфигурации, каждый из которых имеет пропускную способность 50 Мбит/с, измеренный общий goodput на протяжении всего эксперимента устойчиво держится на уровне 48-49 Мбит/с. Это значение лишь незначительно отличается от пропускной способности одного пути (50 Мбит/с) и составляет менее 50% от суммарной емкости двух каналов (100 Мбит/с). Аналогичное наблюдение справедливо и для остальных экспериментов: во всех случаях goodput близок к пропускной способности одного пути и значительно меньше суммарной емкости двух путей.

Интерпретация полученного результата. Наблюдаемое поведение свидетельствует о том, что BBR в условиях пакетной балансировки не способен эффективно использовать совокупную пропускную способность альтернативных путей даже при их полной симметрии. Алгоритм в первом эксперименте ведет себя так, как будто в сети есть только один путь с пропускной способностью около 50 Мбит/с, в остальных экспериментах аналогично.

Причина этого эффекта кроется в механизме оценки пропускной способности узкого места (BtlBw), используемом BBR. Как было описано в разделе 2.1, оценка BtlBw формируется как максимум наблюдаемой скорости доставки (delivery_rate) на скользящем интервале. При пакетной балансировке пакеты распределяются между двумя путями, и скорость доставки ограничивается пропускной способностью каждого отдельного пути. Поскольку пути симметричны, скорость доставки по каждому из них равна пропускной способности одного пути (50 Мбит/с в первом эксперименте). Максимум из этих двух одинаковых значений также равен 50 Мбит/с. Таким образом, BBR оценивает пропускную способность узкого места как 50 Мбит/с, хотя суммарная пропускная способность двух путей составляет 100 Мбит/с. Это фундаментальное ограничение model-based подхода: BBR строит модель одного пути, а не совокупности путей.

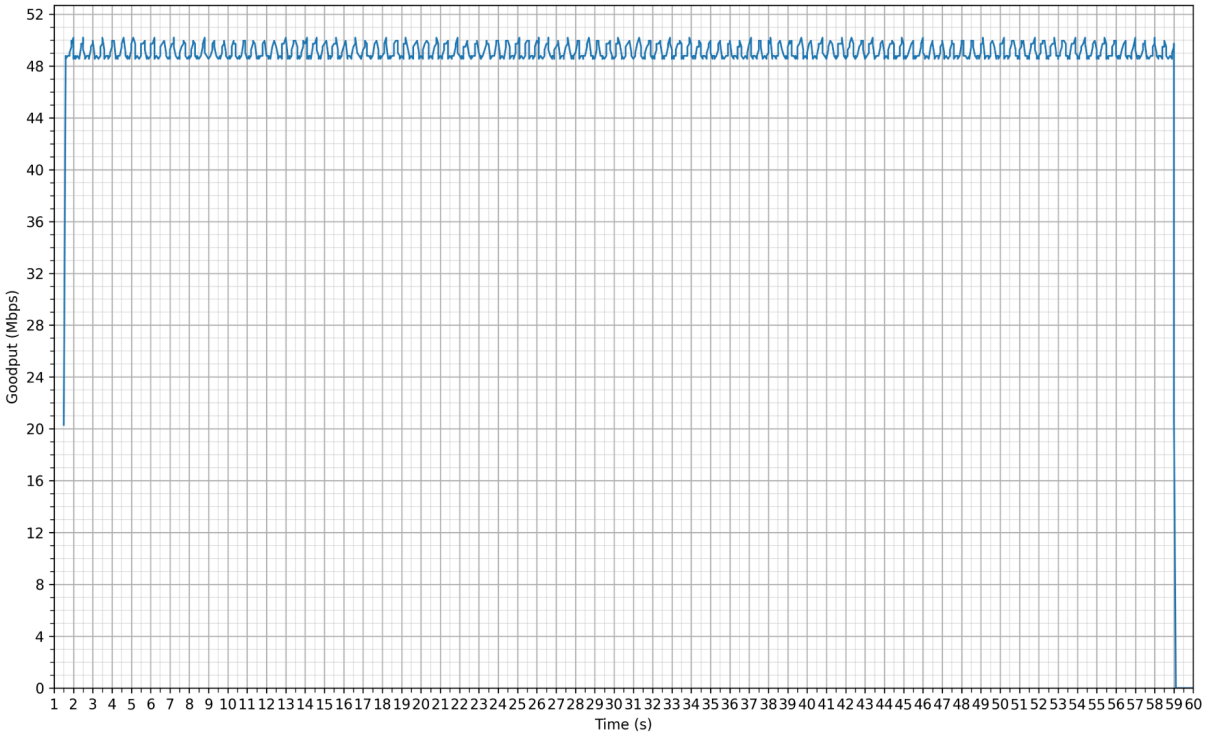

Рисунок 5. Полезная пропускная способность (Goodput) при симметричных альтернативных маршрутах с максимальной скоростью передачи 50 `Мбит/с`.

Постоянство скорости потока. График на рисунке 5 демонстрирует практически идеальную горизонтальную линию без осцилляций и просадок. BBR быстро входит в состояние ProbeBW и остаётся в нём на протяжении всего эксперимента, поддерживая постоянный темп передачи. Отсутствуют как периодические снижения скорости (свойственные состоянию ProbeRTT), так как на протяжении всего времени передачи сохраняется низкий $\mathrm{RTT} = \mathrm{RTprop}$.

Вывод по симметричному сценарию. Эксперимент в симметричной конфигурации демонстрирует, что базовый BBR не способен утилизировать суммарную пропускную способность двух альтернативных путей при пакетной балансировке. Наблюдаемый в первом эксперименте goodput (48-49 Мбит/с) практически соответствует пропускной способности одного пути и составляет лишь половину от доступной совокупной емкости сети (100 Мбит/с). Это ограничение является фундаментальным для BBR и не может быть устранено модификацией порога DupThresh.

4.3. Экспериментальное исследование скорости передачи данных в условиях асимметричных маршрутов

Во второй серии экспериментов исследовано поведение базового алгоритма BBR при наличии асимметрии между параллельными путями. В третьей серии экспериментов исследовано поведение модифицированного алгоритма BBR в тех же условиях. Цель экспериментов — проверить гипотезу о том, что отдельный замедленный путь (с большей задержкой) снижает полезную пропускную способность транспортного соединения, а предложенная модификация позволяет уменьшить негативный эффект за счет сокращения числа ложных повторных передач.

В каждом эксперименте задействовано два альтернативных пути: канал 1 — эталонный с задержкой $y_1 = 10$ мс; канал 2 — замедленный, с задержкой $y_1 = 20$ мс. Пропускная способность задаётся одинаковой для обоих каналов $x_1 = x_2 \in \{50, 100, 200, 500, 1000\}$ Мбит/с. Изменение пропускной способности в экспериментах позволяет оценить влияние дублирующих пакетов при различных значениях окна перегрузки `cwnd`.

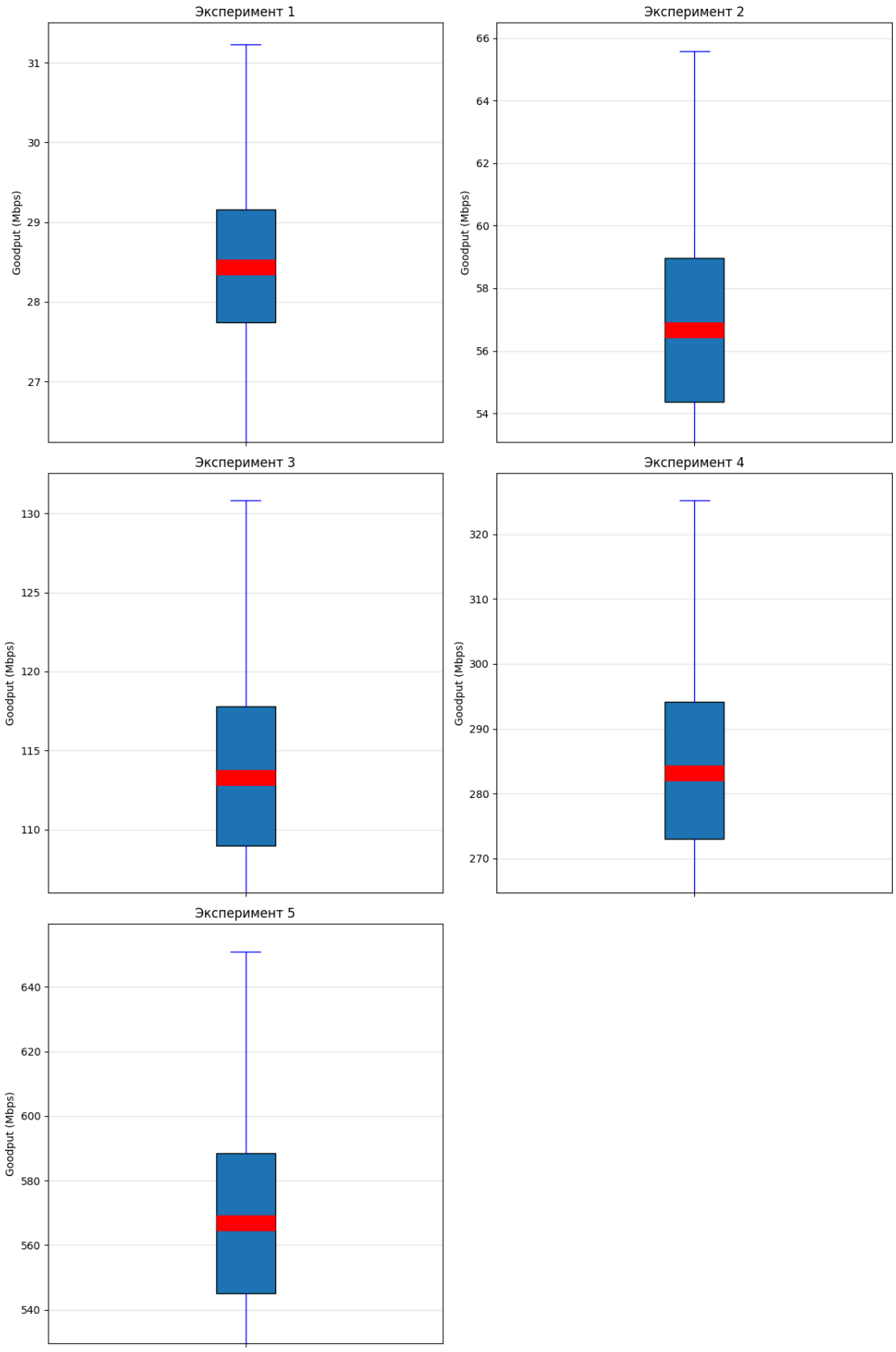

Рисунок 6. Диаграмма размаха полезной пропускной способности базового BBR во времени при асимметричных альтернативных путях с пропускной способностью: 1 — 50 Мбит/с, 2 — 100 Мбит/с, 3 — 200 Мбит/с, 4 — 500 Мбит/с, 5 — 1 Гбит/с. Задержка на первом пути 10 мс, на втором пути — 20 мс.

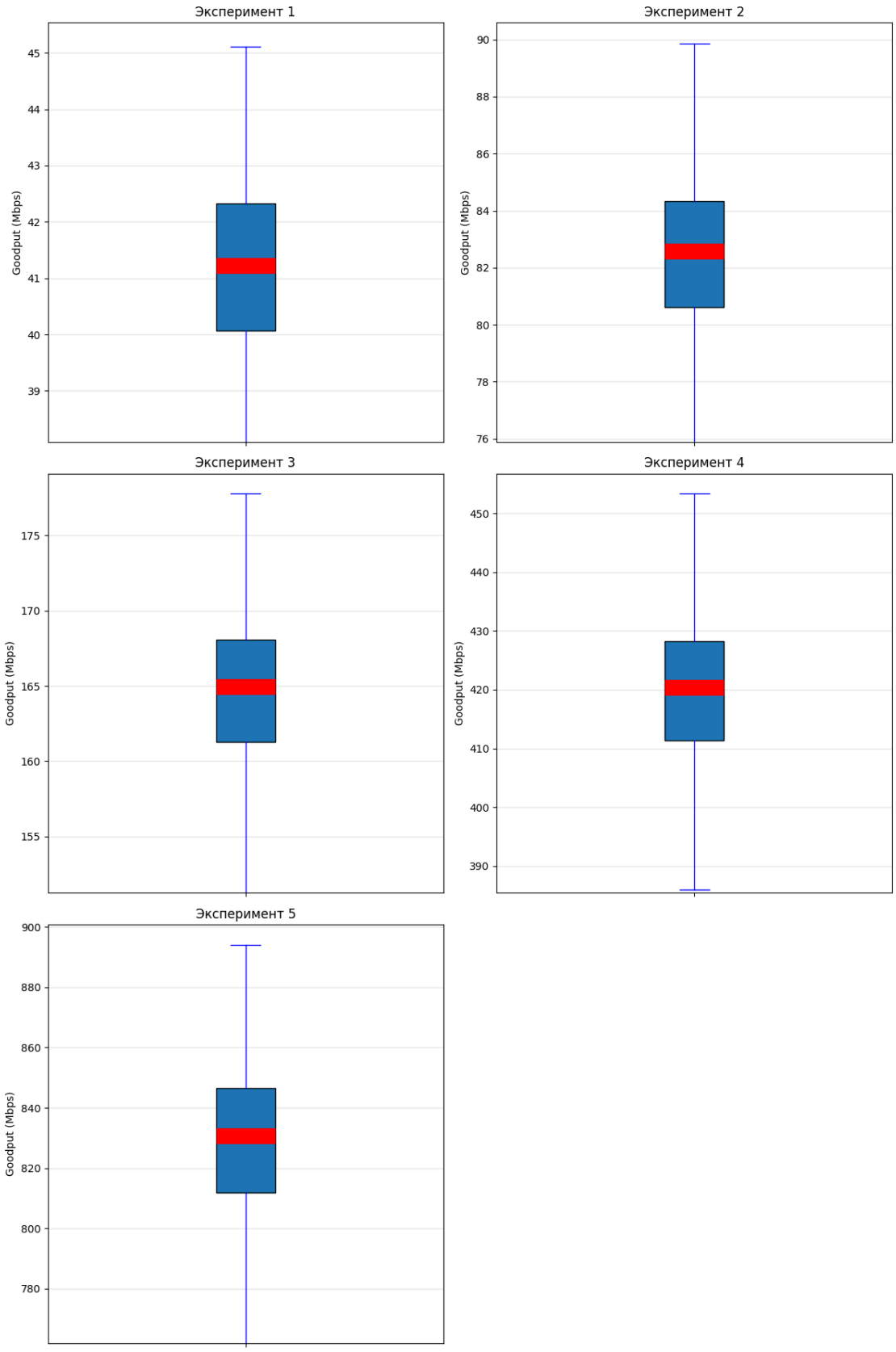

Рисунок 7. Диаграмма размаха полезной пропускной способности модифицированного BBR во времени при асимметричных альтернативных путях с пропускной способностью: 1 — 50 Мбит/с, 2 — 100 Мбит/с, 3 — 200 Мбит/с, 4 — 500 Мбит/с, 5 — 1 Гбит/с. Задержка на первом пути 10 мс, на втором пути — 20 мс.

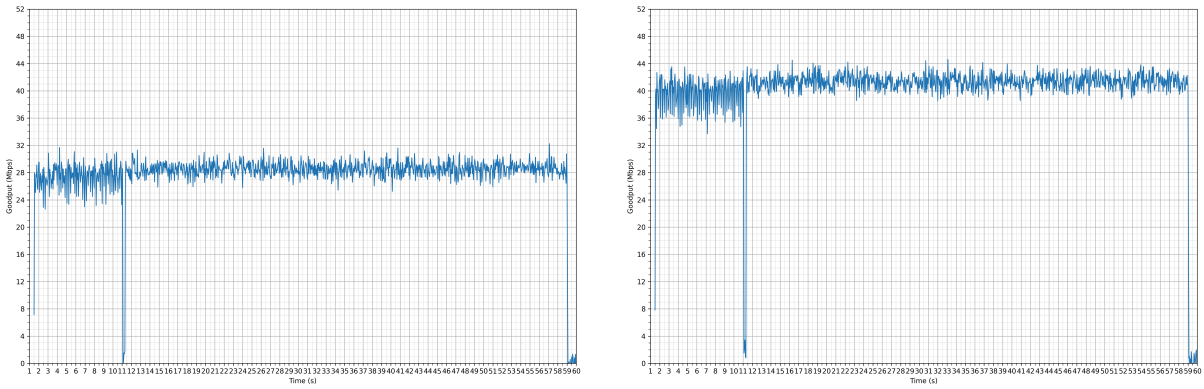

Рисунок 8. Полезная пропускная способность (Goodput) при асимметричных альтернативных путях с пропускной способностью 50 Мбит/с (задержка на 1 пути 10 мс, на 2 пути 20 мс): слева — управление базовым BBR, справа — модифицированным BBR.

Результаты экспериментов. В конфигурации с базовым BBR (Рисунок 6) среднее значение полезной пропускной способности равно 28.3 Мбит/с для первого эксперимента (пропускная способность путей 50 Мбит/с), 56.8 Мбит/с для второго (100 Мбит/с), 114.1 Мбит/с для третьего (200 Мбит/с), 283.7 Мбит/с для четвертого (500 Мбит/с), и 564.4 Мбит/с для пятого эксперимента (1 Гбит/с). Сравнение с симметричным случаем (48.2, 97.1, 195, 489, 988 Мбит/с соответственно) показывает снижение goodput на 41%, 41%, 42%, 42% и 43% соответственно. Во всех конфигурациях наблюдается характерный переходный процесс в начале передачи: в первые 10 секунд происходит кратковременное падение скорости, после чего следует фаза освобождения буфера, и затем соединение выходит на установившийся режим с указанными выше значениями goodput. Это поведение связано с тем, что в начале передачи BBR находится в состоянии StartUp, где агрессивно увеличивает скорость. Когда соединение переходит в ProbeBW, из-за переупорядочивания возникают ложные повторные передачи, и goodput падает до установившегося уровня.

В конфигурации с модифицированным BBR (Рисунок 7) среднее значение полезной пропускной способности составляет 41.2 Мбит/с для первого эксперимента, 82.5 Мбит/с для второго, 165 Мбит/с для третьего, 421 Мбит/с для четвертого, и 832.4 Мбит/с для пятого. По сравнению с базовым BBR, улучшение составляет 46%, 45%, 45%, 48% и 48% соответственно. Однако по сравнению с симметричным случаем все еще наблюдается снижение: модифицированный BBR достигает 85%, 85%, 85%, 86% и 84% от симметричного goodput соответственно. В экспериментах также наблюдается начальный переходный процесс: на первой секунде пропускная способность падает, чтобы очистить буфер в узком месте и обновить значение `RTprop`, после чего скорость потока восстанавливается.

Вывод по сценарию с базовым алгоритмом. Эксперименты демонстрируют, что BBR чувствителен к асимметрии задержек при пакетной балансировке. Увеличение задержки на одном из путей с 10 мс до 20 мс (при сохранении одинаковой пропускной способности) приводит к пропорциональному снижению установившейся полезной пропускной способности примерно на 40-43% во всем диапазоне пропускных способностей. Причина деградации заключается в том, что из-за сильного переупорядочивания пакетов

отправитель повторно посылает пакеты, попавшие на маршрут с большей задержкой. Эти пакеты отправляются повторно до тех пор, пока не попадут на эталонный маршрут (это происходит из-за стандартного механизма `Fast Retransmit`, который повторно отправляет пакет при получении трех дублирующих ACK), чтобы их ACK пришел в нужном порядке или пока не дойдет ACK первого из дублирующихся пакетов. Таким образом, поток занимает оба маршрута множеством дубликатов пакетов, полоса пропускания заполняется ненужными данными и скорость полезной передачи падает.

Вывод по сценарию с модифицированным алгоритмом. Модифицированный BBR демонстрирует повышение полезной пропускной способности во всех экспериментальных конфигурациях по сравнению с базовым алгоритмом в условиях асимметричных путей. Графики на рисунке 8 наглядно иллюстрируют это различие: установившийся уровень goodput для модифицированной версии (правый график) заметно выше, чем для базовой (левый график), при том, что общий характер передачи (начальный переходный процесс с последующей стабилизацией) сохраняется.

Выводы по сценарию с повышенной задержкой.

Полученные результаты подтверждают гипотезу, сформулированную в разделе 3: увеличение порога срабатывания механизма `Fast Retransmit` до значения $\max(\mathrm{BDP}, 3)$ позволяет сократить долю ложных повторных передач, возникающих из-за переупорядочивания пакетов при асимметрии задержек. Как следствие, в установившемся режиме полезная пропускная способность модифицированного BBR во всех экспериментальных конфигурациях превышает показатели базового алгоритма.

Однако сравнение с симметричным сценарием показывает, что даже после модификации полезная пропускная способность в условиях асимметрии задержек остается ниже значений, достигаемых при симметричных путях (дефицит составляет около 15%). Это расхождение объясняется тем, что предложенная модификация устраняет избыточные повторные передачи, но не решает проблему искажения оценки `BtlBw`: при наличии асимметрии задержек измеряемая максимальная скорость доставки по-прежнему ограничена пропускной способностью отдельного пути, а не их суммой. Кроме того, даже при увеличенном пороге некоторое количество повторных передач все равно происходит, особенно в начале каждого цикла ProbeBW, когда отправитель временно увеличивает темп отправки.

5. Заключение

В ходе выполнения данной выпускной квалификационной работы была достигнута поставленная цель: повышение скорости транспортного соединения за счет модификации алгоритма управления перегрузкой BBR, адаптированной к применению в условиях пакетной балансировки. В соответствии с целью были решены следующие задачи.

1. По результатам проведенного обзора версий BBR (BBRv1, BBRv2, BBRv3) по критериям: наличие реакции на потерю пакетов и реализация в симуляторе NS-3, для дальнейшей модификации, был выбран BBRv1.
2. По результатам проведенного обзора существующих алгоритмов управления перегрузкой, адаптированных к пакетной балансировке (TCP-NCR, LTCP, MSwift) по критериям: сохранение скорости передачи при переупорядочивании пакетов, сохранение скорости передачи при ассиметрии каналов, независимость от стратегии балансировки и применимость к BBR, для модификации BBRv1, был выбран принцип динамического порога дублирующих ACK, использованный в TCP-NCR.
3. Предложена модификация BBR для работы в условиях пакетной балансировки. Предложена замена фиксированного порога срабатывания `Fast Retransmit` (3 дублирующих ACK) на динамическое значение $\mathrm{DupACKThresh} = \max\left(\frac{\mathrm{BDP}}{\mathrm{MSS}}, 3\right)$.
4. Проведено экспериментальное исследование разработанного алгоритма. В симуляторе NS-3 с топологией, включающей два альтернативных пути и пакетную балансировку по стратегии round-robin, выполнены три серии экспериментов: с симметричными путями, с асимметричными путями для базового BBR и с асимметричными путями для модифицированного BBR. Установлено, что:

- В симметричной конфигурации базовый BBR достигает полезной пропускной способности, соответствующей одному пути, и не утилизирует суммарную ёмкость двух путей из-за механизма оценки `BtlBw` как максимума скорости доставки по отдельным путям.
- В асимметричной конфигурации (задержки 10 мс и 20 мс) базовый BBR демонстрирует снижение полезной пропускной способности из-за ложных повторных передач, вызванных переупорядочиванием пакетов.
- Модифицированный BBR в ассиметричной конфигурации показывает повышение полезной пропускной способности по сравнению с базовым.

Дальнейшие направления исследований. Несмотря на достигнутое улучшение, предложенная модификация не решает проблему недостижения суммарной пропускной способности альтернативных путей, что обусловлено фундаментальным свойством BBR оценивать `BtlBw` как максимум по путям. Перспективным направлением является разработка алгоритма, способного оценивать суммарную пропускную способность совокупности путей при пакетной балансировке, возможно, за счёт использования методов зондирования с последующей деконволюцией измерений скорости доставки. Кроме того, представляет интерес исследование поведения модифицированного BBR при большем количестве альтернативных путей (более двух) и при динамическом изменении параметров путей во времени.

Литература

1. Dixit A. et al. On the Impact of Packet Spraying in Data Center Networks // IEEE INFOCOM. 2013. 9 с.
2. Bhandarkar S. et al. Improving the Robustness of TCP to Non-Congestion Events // Texas A&M Universit. 2006. 18 с.
3. S.H. Low J. D. F. Paganini. Internet congestion control // IEEE Control Systems Magazine. 2002. 28 с.
4. Cardwell N. et al. BBR: Congestion-Based Congestion Control // Queue. 2016. 34 с.
5. Gerstein B., Silberstein M., Keslassy I. Making congestion control robust to per-packet load balancing in datacenters // arXiv:2509.07907. 2025. 10 с.
6. Network simulator 3 [Электронный ресурс]. URL: https://www.nsnam.org/ (дата обращения: 27.04.2026).
7. Kleinrock L. Power and deterministic rules of thumb for probabilistic problems in computer communications // IEEE. 1979. 10 с.
8. Zhang S. An Evalution of BBR and its variants // In Proceedings of ACM Conference. 2019. 11 с.
9. Zeynali S. F. Danesh Weyulu. Promises and Potential of BBRv3 // 25th International Conference. 2024. 25 с.
10. Zhang J., Zhang D., Huang K. Improving datacenter throughput and robustness with Lazy TCP over packet spraying // Computer Communications. 2015. 11 с.
11. Vivek Jain M. P. T. Viyom Mittal. Design and Implementation of TCP BBR in ns-3 // WNS3. 2018. 7 с.
12. GitHub проекта [Электронный ресурс]. URL: https://github.com/MikleMasterMind/diplom (дата обращения: 27.04.2026).
13. Степанов Е. П. Исследование методов многопоточной маршрутизации для обеспечения качества сетевого сервиса [Электронный ресурс]. 2022. 114 с. URL: https://rusneb.ru/catalog/000199_000009_011131863 (дата обращения: 11.05.2026).